University of Nottingham
UK | CHINA | MALAYSIA

School of Computer Science

A Level 1 Module COMP1038 Autumn 2024/25

Programming and Algorithms

Instructions:

Time allowed: One Hour

Answer ALL questions

Candidates may complete the front cover of their answer book and sign their desk card. **DO NOT turn the examination paper over until instructed to do so.**

Permitted resources:

Those whose first language is not English may use a standard translation dictionary to translate between that language and English provided that neither language is the subject of this examination.

No calculators are permitted in this examination.

Prohibited resources:

All other dictionaries, including subject specific translation dictionaries and electronic dictionaries. Electronic devices capable of storing and retrieving text.

Exam Papers must not be removed from the examination venue.

Note: Answers

Knowledge classification: Following the Schools' recommendation, the (sub) questions have been classified as follows, using a subset of Bloom's Taxonomy:

K: Knowledge

C: Comprehension

A: Application

Question 1: What is the reason and consequence of memory leaks in C? **[2 Marks]**

Solution:

[Knowledge](Difficulty level: easy) A memory leak in C occurs when a program allocates memory on the heap (using functions like malloc(), calloc(), or realloc()) and fails to release it using free() after it is no longer needed. This causes the allocated memory to remain reserved and inaccessible, even though the program has no way to use it, which eventually exhausts available memory, leading to performance degradation or a system crash.

Marking Scheme: 1 mark for the reason of memory leaks and another 1 mark for its consequences.

Question 2: What is a pointer to a pointer in C? Write a C program that uses a pointer to a pointer to change the value of an integer variable.

[5 Marks]

Solution:

[Knowledge](Difficulty level: easy) A pointer to a pointer is a variable that stores the address of another pointer.

```
#include <stdio.h>
void changeValue(int **ptr) {
    **ptr = 100;
}
int main() {
    int value = 50;
    int *ptr = &value;
    int **dptr = &ptr;
    printf("Before: %d\n", value);
    changeValue(dptr);
    printf("After: %d\n", value);
    return 0;
}
```

Marking Scheme: 1 mark for defining pointer to pointer. 1 mark for declaring an integer pointer variable and assigning the address of a variable to it. 1 mark for declaring a pointer to a pointer variable. 1 mark for assigning the address of the pointer variable to the pointer to a pointer variable. 1 mark for assigning an integer value to the pointer to pointer variable print both the old and new values.

Question 3: Error Identification and Correction

The following C program is intended to read a list of integers from the user, calculate their sum, and print the result. However, it contains several errors. Identify and correct these errors. Provide the corrected code and explain each correction you made.

[4 Marks]

```
1   #include <stdio.h>
2   void main() {
3       int n, i, sum;
4       printf("Enter the number of elements: ");
5       scanf("%d", n);
6       int arr[n];
7       printf("Enter %d integers:\n", n);
8       for(i = 0; i <= n; i++) {
9           scanf("%d", &arr);
10      }
11      sum = 0;
12      for(i = 0; i < n; i++) {
13          sum = arr[i];
14      }
15      printf("Sum of the elements: %d\n", sum);
16  }
```

Solution:

[Comprehension](Difficulty level: medium) Errors Identified:

1. **scanf("%d", n)in line 5 should be scanf("%d", &n) - the address of n is needed.**
2. **The loop for(i = 0; i <= n; i++) in line 8 should be for(i = 0; i < n; i++) - The loop runs out of bounds.**
3. **scanf("%d", &arr) in line 9 should be scanf("%d", &arr[i]) – the array index is missing.**
4. **sum = arr[i] in line 13 should be sum += arr[i] – cumulative summation operation is missing.**

Corrected Code:

```
#include <stdio.h>
int main() {
   int n, i, sum;
   printf("Enter the number of elements: ");
   scanf("%d", &n);
   int arr[n];
   printf("Enter %d integers:\n", n);
   for(i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
   }
```

```
    sum = 0;
    for(i = 0; i < n; i++) {
        sum += arr[i];
    }
    printf("Sum of the elements: %d\n", sum);
    return 0;
}
```

Explanation of Corrections:

1. **Corrected scanf for n: Changed scanf("%d", n) to scanf("%d", &n) in line 5 because scanf requires the address of the variable where it stores the input value.**
2. **Fixed Loop Condition: Changed for(i = 0; i <= n; i++) to for(i = 0; i < n; i++) in line 8 to ensure the loop runs exactly n times without going out of bounds.**
3. **Corrected scanf for array index: Changed for scanf("%d", &arr) to scanf("%d", &arr[i]) in line 9 for storing each input element in each individual location of the array.**
4. **Fixed Cumulative Summation Operation: Changed for sum = arr[i] to sum += arr[i] in line 13 to perform cumulative summation operation of each individual input with the intermittent result.**

Marking Scheme: 0.5 mark for identifying each of the mistakes in the program and 0.5 mark for correction for each of the mistakes and explanations.

Question 4: Array Manipulation and Sorting **[6 Marks]**

Problem:

Write a C program that reads 10 integers into an array, sorts the array in ascending order, and then print the sorted array to the screen.

Instructions:

- Read 10 integers from the user into an array.
- Sort the array in ascending order using any sorting algorithm.
- Print every item in the sorted array.

Solution:

[Comprehension, Application](Difficulty level: medium)

```
#include <stdio.h>
void sort(int arr[], int size) {
   for (int i = 0; i < size - 1; i++) {
      for (int j = 0; j < size - 1 - i; j++) {
         if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }
}
int main() {
   int arr[10];
   printf("Enter 10 integers: ");
   for (int i = 0; i < 10; i++) {
      scanf("%d", &arr[i]);
   }
   sort(arr, 10);
   for (int i = 0; i < 10; i++) {
      printf("%d\t", arr[i]);
   }
   return 0;
}
```

Marking Scheme: 2 marks for successfully implementing each of the instructions.

Question 5: Write a C program that counts the number of English alphabetic characters in a given string. The program should prompt the user to input a string, then iterate through each character to count how many are English letters (both uppercase and lowercase). Finally, it should display the total count of English alphabetic characters. The program must not use any built-in string manipulation and character handling functions. You can assume that the input string will contain no more than 1000 characters.

[8 Marks]

Solution:

[Application](Difficulty level: hard)

```
#include <stdio.h>int main() {
   char str[1000];  // Array to store the input string
    int i = 0, count = 0;  // Initialize variables
   // Ask the user to input a string
   printf("Enter a string: ");
   fgets(str, sizeof(str), stdin);  // Read input string from the user
   // Loop through each character in the string
   while (str[i] != '\0') {
      // Check if the character is an English alphabetic character (A-Z or a-z)
      if ((str[i] >= 'A' && str[i] <= 'Z') || (str[i] >= 'a' && str[i] <= 'z')) {
         count++;  // Increment the count if it's an English character
      }
      i++;  // Move to the next character
   }
   // Print the result
   printf("Number of English characters: %d\n", count);
   return 0;
}
```

Marking Scheme: 1 mark for properly declaring the array. 2 marks for properly taking input. 2 marks for properly writing the loop. 2 marks for properly writing the selection statement for English letter identification. 1 mark for printing the number of characters in the input string.

--- End ---